S35 – Evidencia de Aprendizaje 2

Creación de una Base de Datos Staging

Estudiante

Julian Camilo Franco

julian.franco@est.iudigital.edu.co

Bases de Datos II

Profesor

Antonio Jesús Valderrama

Ingeniería de Software y Datos

Institución Universitaria Digital de Antioquia

29 de Marzo 2026

Introducción

En los sistemas de información modernos, la integración y preparación de datos es un proceso fundamental para la toma de decisiones. En este contexto, las bases de datos tipo *Staging* cumplen un papel clave, ya que permiten almacenar temporalmente información proveniente de sistemas operacionales, facilitando su limpieza, transformación y validación antes de su uso en análisis o procesos de inteligencia de negocios.

En esta actividad trabajamos con la base de datos **Jardinería**, con el objetivo de analizar su estructura y contenido, para posteriormente diseñar e implementar una base de datos *Staging* que permita almacenar una copia estructurada y depurada de los datos. Además, se aplican técnicas básicas de limpieza y validación de datos, así como la generación de respaldos para garantizar la integridad de la información.

Objetivos

Objetivo General

Construir una base de datos *Staging* a partir de la base de datos Jardinería, mediante la implementación de consultas SQL que permitan la extracción, transformación y carga de los datos.

Objetivos específicos

- Analizar la estructura de la base de datos Jardinería.
- Diseñar las tablas correspondientes en la base de datos *Staging*.
- Implementar procesos de carga de datos desde la base origen.
- Validar la correcta migración de la información.
- Generar respaldos (BK) de la base origen y de la base *Staging*.

Planteamiento del Problema

La base de datos Jardinería contiene información relevante sobre clientes, pedidos, productos, empleados, pagos y oficinas. Sin embargo, al trabajar directamente sobre esta base de datos, pueden presentarse problemas relacionados con la calidad de los datos, inconsistencias, valores nulos o datos mal formateados.

Además, en escenarios reales de análisis de datos o integración con otros sistemas, no es recomendable trabajar directamente sobre la base de datos operativa, ya que esto puede afectar su rendimiento o integridad.

Por esta razón, se plantea la necesidad de construir una base de datos *Staging*, que permita:

- Tener una copia controlada de los datos.
- Realizar procesos de limpieza y transformación.
- Validar la información antes de su uso final.

Análisis del problema

Se realizó un análisis de la base de datos Jardinería, identificando las principales entidades:

- Oficina
- Empleado
- Cliente
- Pedido
- Producto
- Detalle de pedido
- Pago
- Categoría de producto

Estas tablas contienen información relevante para el negocio y son necesarias para el proceso de migración.

Se identificó que:

- Existen relaciones entre tablas (claves foráneas).
- Algunos campos pueden contener valores nulos.
- Es necesario aplicar limpieza de datos (espacios, valores vacíos).

Por lo tanto, se definió que todas las tablas principales debían ser trasladadas a la base *Staging*, manteniendo su estructura y relaciones.

Se analizó la relevancia de cada tabla dentro del modelo relacional, identificando que entidades como cliente, pedido y detalle_pedido son fundamentales para representar las operaciones del negocio, mientras que tablas como producto y categoría permiten clasificar la información. Este análisis permitió asegurar que la migración a la base Staging conserva la lógica del negocio y facilita su uso posterior en procesos analíticos.

Además, se determinó que estas tablas debían mantenerse en la base de datos Staging conservando sus relaciones mediante llaves primarias y foráneas, ya que esto garantiza la integridad referencial de los datos. Esto es importante para asegurar que la información trasladada mantenga la coherencia con la base de datos original y pueda ser utilizada posteriormente en procesos analíticos o de integración.

Además del reconocimiento de las entidades principales, se analizó la función de cada tabla dentro del proceso de negocio. La tabla **oficina** permite identificar la ubicación organizacional, mientras que **empleado** define la estructura jerárquica de la empresa. Por su parte, **cliente**, **pedido** y **detalle_pedido** representan las operaciones comerciales, siendo fundamentales para el flujo de ventas.

Asimismo, **producto** y **categoria_producto** permiten clasificar la oferta de la empresa, y la tabla **pago** refleja el componente financiero de las transacciones. A partir de este análisis se concluyó que todas estas entidades son necesarias en la base Staging para conservar la lógica relacional del sistema original y garantizar su uso posterior en procesos analíticos.

Adicionalmente, se identificó que trabajar directamente sobre la base de datos operativa puede generar riesgos como inconsistencias en la información, afectación del rendimiento del sistema y dificultad para realizar procesos de análisis. Por esta razón, la implementación de una base de datos Staging permite aislar estos procesos, garantizando un manejo más controlado, seguro y eficiente de los datos.

Propuesta de la solución:

La solución implementada consiste en la creación de una base de datos llamada **Staging**, en la cual se diseñaron tablas equivalentes a las de la base de datos Jardinería.

Diseño de la Base de datos

- Se crearon tablas con prefijo stg_ para almacenar los datos transformados, manteniendo las claves primarias y relaciones entre tablas mediante llaves foráneas.

Proceso de carga de datos

- Se implementaron consultas INSERT INTO ... SELECT para extraer los datos desde la base de datos Jardinería y cargarlos en la base *Staging*, aplicando transformaciones básicas durante el proceso.

Validación de Datos

Se realizaron consultas para:

- Verificar la cantidad de registros por tabla.
- Comparar los datos entre la base origen y la base staging.
- Detectar posibles valores nulos en claves primarias.

  También se verificó que la cantidad de registros en las tablas de la base Staging coincidiera con la base de datos origen, asegurando que no se presentaran pérdidas de información durante el proceso de carga.

Adicionalmente, se verificó que los respaldos generados (`bk_src_*` y `bk_stg_*`) conservaran la misma cantidad de registros que sus tablas de origen, asegurando la integridad de la información. Estas validaciones permiten confirmar que el proceso de carga se realizó correctamente, sin pérdida de datos y manteniendo la consistencia entre la base de datos origen y la base Staging.

Respaldo de la Información (BK)

Se generaron respaldos tanto de la base de datos origen como de la base staging mediante la creación de tablas con prefijo:

- bk_src_ para la base origen
- bk_stg_ para la base staging

Esto permite conservar una copia de los datos en caso de errores o inconsistencias.

Descripción del proceso ETL

Se implementó un proceso ETL compuesto por:

1- Extracción

Se obtuvieron los datos directamente desde la base de datos Jardinería utilizando consultas SELECT.

2- Transformación

Se aplicaron transformaciones básicas:

- Uso de TRIM para eliminar espacios.
- Uso de NULLIF para manejar valores vacíos.
- Uso de LOWER para estandarizar correos electrónicos.

3- Carga

Los datos fueron insertados en las tablas de la base *Staging* mediante instrucciones INSERT INTO ... SELECT.

4- Validación de Datos

Se realizaron validaciones mediante:

- Conteo de registros (COUNT(*)).
- Comparación entre base origen y staging.
- Verificación de valores nulos en claves primarias.

Estas validaciones permiten asegurar que el proceso de carga se realizó correctamente, evitando pérdida de información y garantizando que los datos en la base Staging reflejan fielmente la base de datos origen.

Este proceso garantiza que los datos sean trasladados de manera controlada, asegurando su calidad y consistencia antes de ser utilizados en procesos posteriores de análisis o integración.

Respaldo de información (BK)

Se implementaron dos tipos de respaldo:

- **BK de la base origen:** tablas bk_src_*
- **BK de la base staging:** tablas bk_stg_*

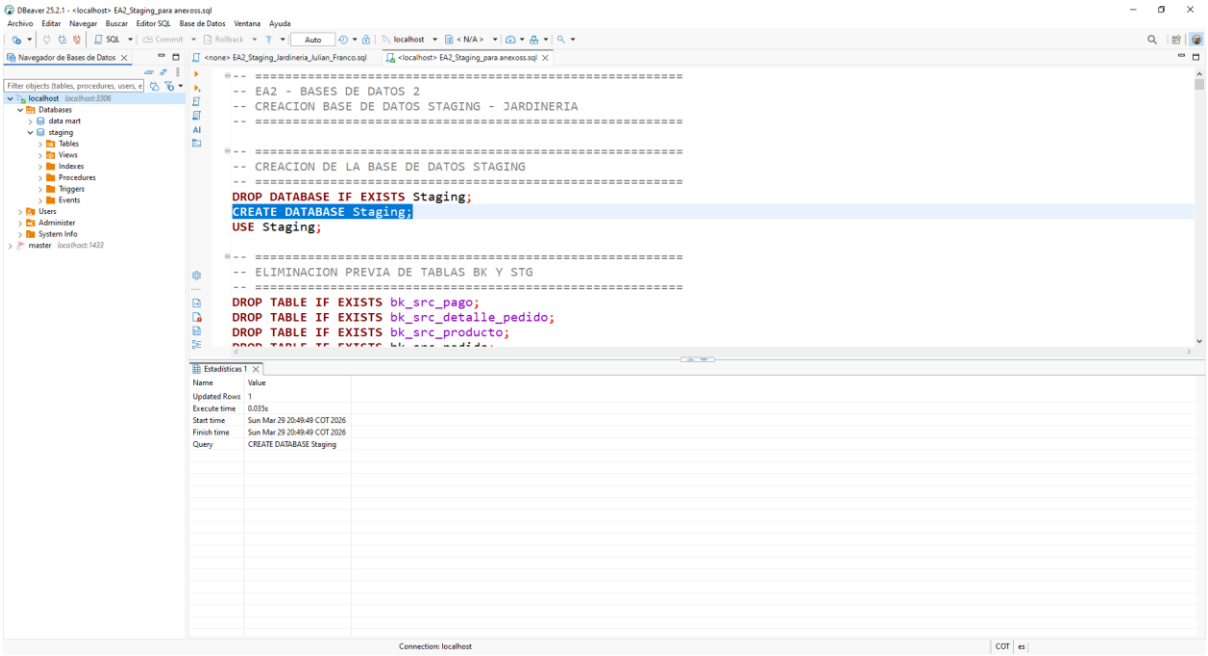
Esto garantiza la disponibilidad de la información ante cualquier fallo.

Anexos

Se adjunta:

- Anexo A. Script SQL de creación de la base de datos Staging.

```sql
DROP DATABASE IF EXISTS Staging;
CREATE DATABASE Staging;
USE Staging;
```




```sql
CREATE TABLE stg_oficina (
ID_oficina INT NOT NULL,
Descripcion VARCHAR(10) NOT NULL,
ciudad VARCHAR(30) NOT NULL,
pais VARCHAR(50) NOT NULL,
region VARCHAR(50) DEFAULT NULL,
codigo_postal VARCHAR(10) NOT NULL,
telefono VARCHAR(20) NOT NULL,
```

```sql
linea_direccion1 VARCHAR(50) NOT NULL,
linea_direccion2 VARCHAR(50) DEFAULT NULL,
PRIMARY KEY (ID_oficina)
);

CREATE TABLE stg_empleado (
ID_empleado INT NOT NULL,
nombre VARCHAR(50) NOT NULL,
apellido1 VARCHAR(50) NOT NULL,
apellido2 VARCHAR(50) DEFAULT NULL,
extension VARCHAR(10) NOT NULL,
email VARCHAR(100) NOT NULL,
ID_oficina INT NOT NULL,
ID_jefe INT DEFAULT NULL,
puesto VARCHAR(50) DEFAULT NULL,
PRIMARY KEY (ID_empleado)
);

CREATE TABLE stg_categoria_producto (
Id_Categoria INT NOT NULL,
Desc_Categoria VARCHAR(50) NOT NULL,
descripcion_texto TEXT,
descripcion_html TEXT,
imagen VARCHAR(256),
PRIMARY KEY (Id_Categoria)
);

CREATE TABLE stg_cliente (
ID_cliente INT NOT NULL,
nombre_cliente VARCHAR(50) NOT NULL,
nombre_contacto VARCHAR(30) DEFAULT NULL,
apellido_contacto VARCHAR(30) DEFAULT NULL,
```

```sql
telefono VARCHAR(15) NOT NULL,
fax VARCHAR(15) NOT NULL,
linea_direccion1 VARCHAR(50) NOT NULL,
linea_direccion2 VARCHAR(50) DEFAULT NULL,
ciudad VARCHAR(50) NOT NULL,
region VARCHAR(50) DEFAULT NULL,
pais VARCHAR(50) DEFAULT NULL,
codigo_postal VARCHAR(10) DEFAULT NULL,
ID_empleado_rep_ventas INT DEFAULT NULL,
limite_credito DECIMAL(15,2) DEFAULT NULL,
PRIMARY KEY (ID_cliente)
);

CREATE TABLE stg_pedido (
ID_pedido INT NOT NULL,
fecha_pedido DATE NOT NULL,
fecha_esperada DATE NOT NULL,
fecha_entrega DATE DEFAULT NULL,
estado VARCHAR(15) NOT NULL,
comentarios TEXT,
ID_cliente INT NOT NULL,
PRIMARY KEY (ID_pedido)
);

CREATE TABLE stg_producto (
ID_producto VARCHAR(15) NOT NULL,
nombre VARCHAR(70) NOT NULL,
Categoria INT NOT NULL,
dimensiones VARCHAR(25) DEFAULT NULL,
proveedor VARCHAR(50) DEFAULT NULL,
descripcion TEXT,
cantidad_en_stock SMALLINT NOT NULL,
```

```
precio_venta DECIMAL(15,2) NOT NULL,
precio_proveedor DECIMAL(15,2) DEFAULT NULL,
PRIMARY KEY (ID_producto)
);

CREATE TABLE stg_detalle_pedido (
ID_pedido INT NOT NULL,
ID_producto VARCHAR(15) NOT NULL,
cantidad INT NOT NULL,
precio_unidad DECIMAL(15,2) NOT NULL,
numero_linea SMALLINT NOT NULL,
PRIMARY KEY (ID_pedido, ID_producto),
UNIQUE KEY uk_stg_detalle_linea (ID_pedido,
numero_linea)
);

CREATE TABLE stg_pago (
ID_cliente INT NOT NULL,
forma_pago VARCHAR(40) NOT NULL,
id_transaccion VARCHAR(50) NOT NULL,
fecha_pago DATE NOT NULL,
total DECIMAL(15,2) NOT NULL,
PRIMARY KEY (ID_cliente, id_transaccion)
);
```

- Anexo B. Script de carga y transformación de datos.

A continuación, se presenta el script utilizado para la carga de datos desde la base de datos origen hacia la base de datos Staging, aplicando procesos básicos de limpieza como TRIM, LOWER y NULLIF:

```
INSERT INTO stg_oficina (
```

```
ID_oficina,
Descripcion,
ciudad,
pais,
region,
codigo_postal,
telefono,
linea_direccion1,
linea_direccion2
)
SELECT
o.ID_oficina,
TRIM(o.Descripcion),
TRIM(o.ciudad),
TRIM(o.pais),
NULLIF(TRIM(o.region), ''),
TRIM(o.codigo_postal),
TRIM(o.telefono),
TRIM(o.linea_direccion1),
NULLIF(TRIM(o.linea_direccion2), '')
FROM jardineria.oficina o;

INSERT INTO stg_empleado (
ID_empleado,
nombre,
apellido1,
apellido2,
extension,
```

```
email,
ID_oficina,
ID_jefe,
puesto
)
SELECT
e.ID_empleado,
TRIM(e.nombre),
TRIM(e.apellido1),
NULLIF(TRIM(e.apellido2), ''),
TRIM(e.extension),
LOWER(TRIM(e.email)),
e.ID_oficina,
e.ID_jefe,
NULLIF(TRIM(e.puesto), '')
FROM jardineria.empleado e;

INSERT INTO stg_categoria_producto (
Id_Categoria,
Desc_Categoria,
descripcion_texto,
descripcion_html,
imagen
)
SELECT
cp.Id_Categoria,
TRIM(cp.Desc_Categoria),
cp.descripcion_texto,
```

```
cp.descripcion_html,
cp.imagen
FROM jardineria.Categoria_producto cp;

INSERT INTO stg_cliente (
ID_cliente,
nombre_cliente,
nombre_contacto,
apellido_contacto,
telefono,
fax,
linea_direccion1,
linea_direccion2,
ciudad,
region,
pais,
codigo_postal,
ID_empleado_rep_ventas,
limite_credito
)
SELECT
c.ID_cliente,
TRIM(c.nombre_cliente),
NULLIF(TRIM(c.nombre_contacto), ''),
NULLIF(TRIM(c.apellido_contacto), ''),
TRIM(c.telefono),
TRIM(c.fax),
TRIM(c.linea_direccion1),
```

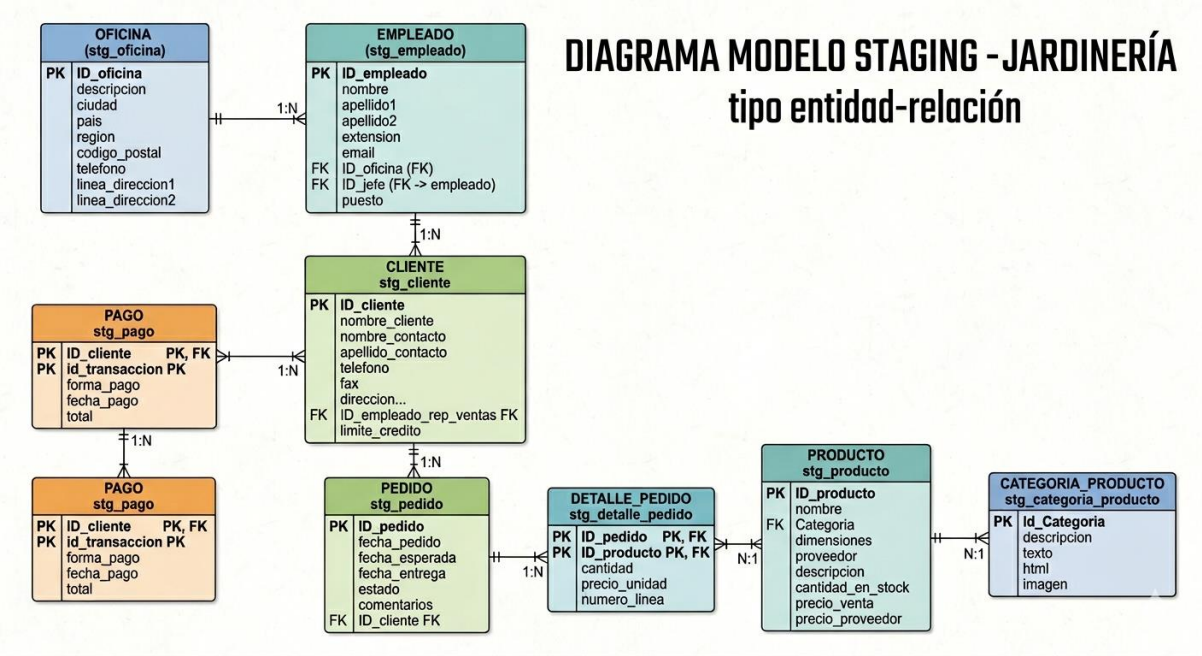
```sql
NULLIF(TRIM(c.linea_direccion2), ''),
TRIM(c.ciudad),
NULLIF(TRIM(c.region), ''),
NULLIF(TRIM(c.pais), ''),
NULLIF(TRIM(c.codigo_postal), ''),
c.ID_empleado_rep_ventas,
c.limite_credito
FROM jardineria.cliente c;

INSERT INTO stg_pedido (
ID_pedido,
fecha_pedido,
fecha_esperada,
fecha_entrega,
estado,
comentarios,
ID_cliente
)
SELECT
p.ID_pedido,
p.fecha_pedido,
p.fecha_esperada,
p.fecha_entrega,
TRIM(p.estado),
p.comentarios,
p.ID_cliente
FROM jardineria.pedido p;

INSERT INTO stg_producto (
```

```
ID_producto,
nombre,
Categoria,
dimensiones,
proveedor,
descripcion,
cantidad_en_stock,
precio_venta,
precio_proveedor
)
SELECT
pr.ID_producto,
TRIM(pr.nombre),
pr.Categoria,
NULLIF(TRIM(pr.dimensiones), ''),
NULLIF(TRIM(pr.proveedor), ''),
pr.descripcion,
pr.cantidad_en_stock,
pr.precio_venta,
pr.precio_proveedor
FROM jardineria.producto pr;

INSERT INTO stg_detalle_pedido (
ID_pedido,
ID_producto,
cantidad,
precio_unidad,
numero_linea
```

```
)
SELECT
dp.ID_pedido,
dp.ID_producto,
dp.cantidad,
dp.precio_unidad,
dp.numero_linea
FROM jardineria.detalle_pedido dp;

INSERT INTO stg_pago (
ID_cliente,
forma_pago,
id_transaccion,
fecha_pago,
total
)
SELECT
pa.ID_cliente,
TRIM(pa.forma_pago),
TRIM(pa.id_transaccion),
pa.fecha_pago,
pa.total
FROM jardineria.pago pa;
```

Modelo de datos Staging

A continuación, se presenta el diagrama entidad-relación de la base de datos Staging, el cual mantiene la estructura y relaciones de la base de datos original Jardinería, asegurando la integridad referencial de los datos.



Este modelo conserva las relaciones 1:N entre las entidades principales, permitiendo mantener la consistencia de los datos durante el proceso ETL.

Conclusiones

El desarrollo de esta actividad me permitió comprender la importancia de las bases de datos tipo *Staging* dentro de los procesos de integración de datos. A partir del análisis de la base de

datos Jardinería, se logró identificar las entidades principales y diseñar una estructura equivalente en una nueva base de datos, manteniendo la integridad de la información.

Asimismo, se implementaron procesos de extracción, transformación y carga (ETL), aplicando técnicas básicas de limpieza de datos que contribuyen a mejorar la calidad de la información. La validación realizada mediante consultas permitió comprobar que los datos fueron migrados correctamente, sin pérdida de registros.

Finalmente, la generación de respaldos tanto de la base de datos origen como de la base *Staging* garantiza la disponibilidad de la información y permite mitigar riesgos ante posibles fallos. Este proceso evidencia la importancia de trabajar con entornos controlados antes de utilizar los datos en análisis o sistemas de apoyo a la decisión

Bibliografía

-IU DIGITAL DE ANTIOQUIA

Modulo 3

Diseños de procesos ETL

https://iudigital.instructure.com/courses/25656/pages/tema-1-introduccion-al-diseno-de-procesos-etl?module_item_id=1657776

-Kimball, R. & Ross, M. (2013). The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling.

-Oracle. MySQL Documentation.

- OpenAI. (2026). *ChatGPT (modelo GPT-5.3)* [Modelo de lenguaje].

-Google. (2026). *Gemini* [Modelo de inteligencia artificial].